

✓ Camada Convolutacional

Como vimos, as camadas densas tem severos problemas, principalmente para trabalhar com imagens, pois a dimensionalidade da entrada é muito grande o que implica num alto número de parâmetros para se otimizar.

Contornando essa situação, temos as camadas convolucionais. Ao longo do tempo, vários tipos de camadas convolucionais foram criadas. Nesta aula veremos algumas dessas camadas.

```
import time, os, sys, numpy as np
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F

from torch import optim

import time, os, sys, numpy as np

# Test if GPU is available, if not, use cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
n = torch.cuda.device_count()
devices_ids = list(range(n))
print(device)
```

cuda

```
def load_data_fashion_mnist(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Fashion-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.FashionMNIST(root=root, train=True, download=True, transform=transformer)
    mnist_test = torchvision.datasets.FashionMNIST(root=root, train=False, download=True, transform=transformer)
    num_workers = 0 if sys.platform.startswith('win32') else 4

    train_iter = torch.utils.data.DataLoader(mnist_train,
                                              batch_size, shuffle=True,
                                              num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(mnist_test,
                                            batch_size, shuffle=False,
                                            num_workers=num_workers)

    return train_iter, test_iter

# funções básicas
def _get_batch(batch):
    """Return features and labels on ctx."""
    features, labels = batch
    if labels.type() != features.type():
        labels = labels.type(features.type())
    return (torch.nn.DataParallel(features, device_ids=devices_ids),
            torch.nn.DataParallel(labels, device_ids=devices_ids), features.shape[0])

# Função usada para calcular acurácia
def evaluate_accuracy(data_iter, net, loss):
    """Evaluate accuracy of a model on the given data set."""

    acc_sum, n, l = torch.Tensor([0]), 0, 0
    net.eval()
    with torch.no_grad():
        for X, y in data_iter:
            #y = y.astype('float32')
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l += loss(y_hat, y).sum()
            acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]

    return acc_sum.item() / n, l.item() / len(data_iter)

# Função usada no treinamento e validação da rede
def train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
                  num_epochs):
```

```

print('training on', device)
for epoch in range(num_epochs):
    net.train()
    train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0, time.time()
    for X, y in train_iter:
        X, y = X.to(device), y.to(device)
        y_hat = net(X)
        trainer.zero_grad()
        l = loss(y_hat, y).sum()
        l.backward()
        trainer.step()
        train_l_sum += l.item()
        train_acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
        n += y.size()[0]
    test_acc, test_loss = evaluate_accuracy(test_iter, net, loss)
    print('epoch %d, train loss %.4f, train acc %.3f, test loss %.4f, '
          'test acc %.3f, time %.1f sec'
          % (epoch + 1, train_l_sum / len(train_iter), train_acc_sum / n, test_loss,
             test_acc, time.time() - start))

def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        m.weight.data.normal_(0.0, 0.01)
    if classname.find('Linear') != -1:
        m.weight.data.normal_(0.0, 0.01)

```

✓ Camada Convolutacional

A [camada convolutacional](#) é considerado o principal módulo das rede convolucionais (*ConvNet*) pois é responsável por fazer a maior parte do trabalho, ou seja, aprender filtros que extraem características.

Tecnicamente, uma camada convolutacional é composta por um conjunto de filtros (ou *kernels*) aprendíveis (que, na verdade, representam os parâmetros dessa camada). Geralmente, cada um desses filtros é relativamente pequeno (em termos de largura e altura), mas se estende por toda a profundidade dos dados de entrada. Por exemplo, cada filtro em uma primeira camada de uma *ConvNet* geralmente tem tamanho $3 \times h \times w$, onde h e w são altura e largura, respectivamente, e o 3 representa a profundidade dos filtros que, neste caso, são ligados aos canais de cores da entrada (RGB). Num primeiro momento, trabalharemos somente com filtros bidimensionais pois deixaremos de lado a dimensão relacionada aos canais. Durante o *forward*, convolucionamos cada filtro sobre a entrada calculando o produto entre esses valores e gerando, como saída, um mapa de ativação bidimensional que fornece as respostas desse filtro em todas as possíveis posições. Em outras palavras, cada filtro pode ser visto como um neurônio que irá combinar os canais de entradas gerando uma saída. Intuitivamente, a rede aprenderá filtros que são ativados quando encontram algum tipo de características visual interessante, como uma borda na primeira camada, ou eventualmente padrões mais complexos nas camadas mais finais da rede. Cada camada convolutacional é composta por conjuntos inteiro de filtros onde, cada um deles, produzirá um mapa de ativação bidimensional separado. Esses mapas de ativação são empilhados (na profundidade) produzindo a saída final (comumente chamada de *feature maps*).

Formalmente, o procedimento de convolução 2-D, comumente empregado nesse tipo de camada, recebe uma entrada de duas dimensões x e um vetor de peso 2-D K (nesse caso, com tamanho $n \times n$) e os processa da seguinte forma:

$$Y[k, l] = \sum_{i=1}^n \sum_{j=1}^n X(k+i-1, l+j-1) K(i, j)$$

, onde Y é a saída ou *feature map*.

Abaixo temos um exemplo de convolução. Neste caso, a entrada é uma matriz bidimensional com uma altura de 3 e largura de 3 (3×3) e o filtro tem dimensões 2×2 . Este exemplo destaca (em azul) um passo do processo de convolução. Entretanto, como dito anteriormente, a janela (filtro) de convolução percorre toda a entrada ao longo da altura e largura, sempre multiplicando e somando os elementos da entrada pelos valores do filtro.



O exemplo acima temos somente **um** canal de entrada e de saída. Porém, como funciona a convolução com múltiplos canais de entrada e saída?

Múltiplos canais de entrada e saída

No exemplo anterior, tanto os filtros quanto as saídas podem ser vistas como matrizes bidimensionais. Entretanto, quando adicionamos mais canais, entradas, filtros, e saídas passam a ser representadas como matrizes tridimensionais. Por exemplo, cada imagem de entrada RGB tem a forma $3 \times h \times w$, onde h e w são a altura e largura respectivamente.. A dimensão com um tamanho de 3 é referenciada como a dimensão do canal.

Múltiplos Canais de Entrada

Quando os dados de entrada contêm múltiplos canais, precisamos de filtros de convolução com o mesmo número de canais que os dados de entrada, para que ele possamos processar uma convolução. Assumindo que o número de canais para os dados de entrada é c_i , o número de canais dos filtro de convolução também precisa ser c_i . Quando $c_i = 1$, o filtro de convolução é uma matriz bidimensional com dimensões $k_h \times k_w$.

No entanto, quando $c_i > 1$, precisamos de um filtro que contenha uma matriz $k_h \times k_w$ **para cada canal de entrada**. Concatenando todos esses c_i arrays juntos geramos um filtro de convolução com dimensões $c_i \times k_h \times k_w$. Como a entrada e o filtro da convolução tem cada um c_i canais, podemos criar uma correspondência entre canais para executar a operação de convolução. Em outras palavras, podemos fazer a convolução da matriz bidimensional da entrada com o *kernel* bidimensional para cada canal, somando os resultados ao longo dos canais c_i , produzindo uma só matriz bidimensional de saída.

Na figura abaixo, demonstramos um exemplo de convolução com dois canais de entrada. As partes sombreadas representam o primeiro elemento de saída, bem como os elementos de entrada e de matriz do kernel usados em sua computação:

$$(1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) + (0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3) = 56.$$



Múltiplos canais de saída

Independentemente do número de canais de entrada, até agora nós sempre acabamos com um canal de saída. No entanto, aumentar o número de canais (neurônios) em cada camada implica em aumentar a poder de representação daquela camada. Nas arquiteturas de redes neurais mais populares, na verdade aumentamos a dimensão do canal à medida que avançamos na rede neural, geralmente diminuindo a resolução.

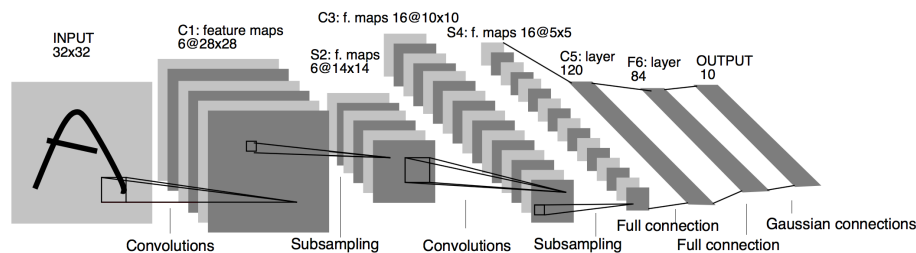
Denote por c_i e c_o o número de canais de entrada e saída, respectivamente, e deixe que k_h e k_w sejam a altura e a largura do filtro convolutacional. Para obter uma saída com múltiplos canais, podemos criar uma matriz de kernel de forma $c_i \times k_h \times k_w$ para cada canal de saída c_o (que também pode ser visto como neurônio). Concatenamos na dimensão do canal de saída, para que o filtro de convolução tenha resolução final de $c_o \times c_i \times k_h \times k_w$. Nas operações de convolução, o resultado em cada canal de saída é calculado a partir do filtro de convolução correspondente a esse canal de saída e recebe a entrada de todos os canais na matriz de entrada.

Para ter uma visão mais ampla da convolução em múltiplos canais de entrada e saída, acesse esse [site](#) e procurem pela gif relacionada ao tema.

Pytorch e o caso de estudo LeNet-5

Frameworks modernos implementam camadas convolucionais de forma fácil e intuitiva. No Pytorch, a [camada de convolução](#) tem alguns dos parâmetros que vimos anteriormente na sua declaração.

Vamos implementar uma rede baseada na [LeNet-5](#) e entender cada parâmetro. A rede tem essa arquitetura (ignorem, nesse primeiro momento, os *subsamplings*):



As camadas com a letra C, são convoluções. Camadas que começam com a letra S, são *subsampling* e devem ser ignoradas neste primeiro momento. Já camada com início F, são *fully-connected*. Abaixo, uma tabela que compila toda a configuração da rede.

	Layer	Feature Map	Size	Kernel Size	Stride	Activation
	Input	Image	1	32x32	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

Abaixo, recriamos a rede no Pytorch **sem as camadas de *subsampling***.

```
# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 20, 0.1, 128, 0.000001

# rede baseada na LeNet-5
net = nn.Sequential(
    nn.Conv2d(in_channels=1,out_channels=6, kernel_size=5),      # entrada: 1 canal e saída: 6 canais
    nn.Tanh(),
    # vamos ignorar o subsampling/pooling por agora
    nn.Conv2d(in_channels=6,out_channels= 16, kernel_size=5),   # entrada: 6 canais e saída: 16 canais
    nn.Tanh(),
    # vamos ignorar o subsampling/pooling por agora
    nn.Conv2d(in_channels=16, out_channels=120, kernel_size=5), # entrada: 16 canais e saída: 120 canais
    nn.Tanh(),
    # vamos ignorar o subsampling/pooling por agora
    nn.Flatten(), # lineariza formando um vetor                # entrada: 120 canais e saída: linear # algumas vezes, impleme
    nn.Linear(48000, 84),
    nn.Tanh(),
    nn.Linear(84, 10)
)

# Sending model to device
net.to(device)

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_fashion_mnist(batch_size, resize=32)

# trainer do pytorch
trainer = optim.SGD(net.parameters(), lr=0.01, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss, num_epochs)
```

```
100%|██████████| 26.4M/26.4M [00:02<00:00, 12.9MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 202kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.79MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 27.0MB/s]training on cuda
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:432: UserWarning: This DataLoader will create 4 worker
  self.check_worker_number_rationality()
epoch 1, train loss 0.8660, train acc 0.709, test loss 0.6285, test acc 0.767, time 12.3 sec
epoch 2, train loss 0.5658, train acc 0.791, test loss 0.5462, test acc 0.800, time 13.3 sec
epoch 3, train loss 0.5038, train acc 0.818, test loss 0.5170, test acc 0.814, time 11.2 sec
epoch 4, train loss 0.4668, train acc 0.833, test loss 0.4911, test acc 0.819, time 12.2 sec
epoch 5, train loss 0.4410, train acc 0.842, test loss 0.4693, test acc 0.827, time 16.0 sec
epoch 6, train loss 0.4211, train acc 0.851, test loss 0.4396, test acc 0.843, time 11.5 sec
epoch 7, train loss 0.4051, train acc 0.857, test loss 0.4314, test acc 0.844, time 11.4 sec
epoch 8, train loss 0.3903, train acc 0.862, test loss 0.4165, test acc 0.849, time 11.9 sec
epoch 9, train loss 0.3778, train acc 0.865, test loss 0.4095, test acc 0.851, time 12.0 sec
epoch 10, train loss 0.3676, train acc 0.869, test loss 0.3914, test acc 0.861, time 11.8 sec
epoch 11, train loss 0.3575, train acc 0.873, test loss 0.3864, test acc 0.863, time 11.9 sec
epoch 12, train loss 0.3492, train acc 0.876, test loss 0.3855, test acc 0.861, time 11.8 sec
epoch 13, train loss 0.3395, train acc 0.879, test loss 0.3747, test acc 0.869, time 12.4 sec
epoch 14, train loss 0.3320, train acc 0.881, test loss 0.3768, test acc 0.864, time 12.0 sec
epoch 15, train loss 0.3260, train acc 0.884, test loss 0.3613, test acc 0.871, time 11.8 sec
epoch 16, train loss 0.3174, train acc 0.887, test loss 0.3632, test acc 0.867, time 12.0 sec
epoch 17, train loss 0.3106, train acc 0.889, test loss 0.3734, test acc 0.861, time 11.3 sec
epoch 18, train loss 0.3048, train acc 0.891, test loss 0.3505, test acc 0.874, time 11.4 sec
epoch 19, train loss 0.2994, train acc 0.892, test loss 0.3590, test acc 0.870, time 12.1 sec
epoch 20, train loss 0.2930, train acc 0.896, test loss 0.3430, test acc 0.875, time 11.7 sec
```

➤ Hiper-parâmetros: *Padding* e *Stride*

No exemplo anterior, a entrada tinha dimensões 6×8 e o filtro de convolução 1×2 . O processamento da convolução produziu, então, uma saída com uma resolução 6×7 . Essa diferença da resolução é motivada pelo próprio processamento da convolução e da sua forma de lidar com as extremidades da imagem. Em geral, assumindo que a entrada tem tamanho $n_h \times n_w$ e o filtro de convolução tem dimensões $k_h \times k_w$, então o tamanho da saída pode ser calculado da seguinte forma:

$$(n_h - k_h + 1) \times (n_w - k_w + 1)$$

Neste caso, as dimensões da saída é determinada pelos tamanhos da entrada e do filtro de convolução.

Em alguns casos, podemos incorporar técnicas comum de processamento de imagem (como *Padding* e *Stride*) que afetam diretamente o tamanho da saída:

- Em geral, como filtros geralmente têm dimensões maiores que 1, após muitas convoluções sucessivas, a saída termina ficando muito menor do que a entrada. Por exemplo, imagine que uma imagem de entrada com 240×240 pixels seja processada por 10 camadas de convoluções 5×5 . Neste caso, a imagem inicial será reduzida para uma saída de 200×200 pixels, ou seja, 30% da imagem original é eliminada e, com ela, informações interessante próxima das extremidades da imagem de entrada. *Padding* lida com esse problema.
- Em alguns casos, como quando a entrada tem uma resolução muito grande, queremos reduzir drasticamente a resolução da imagem durante o seu processamento. *Strides* podem ajudar nesses casos.

Padding

Como descrito acima, um problema complicado ao se trabalhar com camadas convolucionais é a perda de pixels (e, consequentemente, informação) na extremidade da imagem. Como normalmente usamos filtros pequenos, a perda num geral é pequena. Entretanto, ela se torna maior à medida que aplicamos várias camadas convolucionais sucessivas. Uma solução direta para esse problema é adicionar pixels extras ao redor da imagem de entrada, de forma a aumentar o tamanho efetivo da imagem. Esse processo é conhecido como *padding*. Normalmente, definimos os valores desses pixels extras como 0 (*zero-padding*). Abaixo, temos um exemplo visual de uma entrada 3×3 com *padding* de tamanho 1 em todos os lados sendo processada por um *kernel* 2×2 .

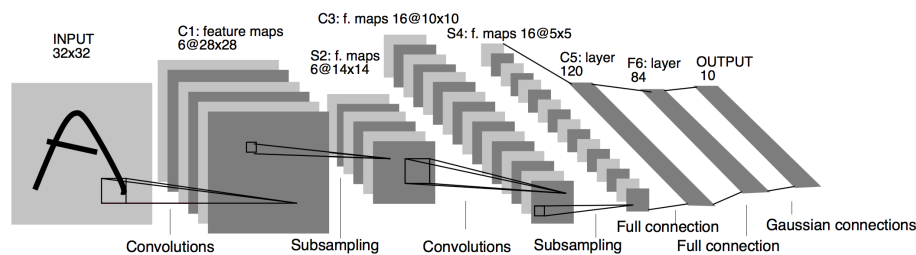


Para ficar ainda mais claro o funcionamento do *padding*, implementamos esse processo usando o trecho de código abaixo. Neste caso, a entrada, de tamanho 3×3 , tem sua resolução aumentada 5×5 usando *padding*. Dessa forma, a saída correspondente é também aumentada para 4×4 .

↳ 7 células ocultas

Continuação: Pytorch e o caso de estudo LeNet-5

Vamos agora definir explicitamente o *padding* e *stride* em cada camada convolutacional. Como, por padrão, o *padding* e *stride* dessa camada é 0 e 1, o nosso resultado ainda não vai mudar. Entretanto, agora podemos calcular a saída exata de cada camada.



	Layer	Feature Map	Size	Kernel Size	Stride	Activation
	Input	Image	1	32x32	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

```
# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 10, 0.1, 128, 0.000001

# rede baseada na LeNet-5
net = nn.Sequential(
    nn.Conv2d(in_channels=1,out_channels=6, kernel_size=5, stride=1, padding=0), # entrada: (b, 1, 32, 32) e
    nn.Tanh(), # entrada: (b, 6, 28, 28) e
    nn.Conv2d(in_channels=6,out_channels=16, kernel_size=5, stride=1, padding=0), # entrada: (b, 16, 24, 24) e
    nn.Tanh(), # entrada: (b, 120, 20, 20)
    nn.Flatten(), # lineariza formando um vetor # entrada: (b, 48000) e saída
    nn.Linear(48000, 84),
    nn.Tanh(),
```

```
nn.Linear(84, 10)
)

# Sending model to device
net.to(device)

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_fashion_mnist(batch_size, resize=32)

# trainer do gluon
trainer = optim.SGD(net.parameters(), lr=lr, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)
```

entrada: (b, 84) e saída:

```
training on cuda
epoch 1, train loss 0.5620, train acc 0.795, test loss 0.4596, test acc 0.829, time 11.6 sec
epoch 2, train loss 0.3997, train acc 0.856, test loss 0.3887, test acc 0.858, time 12.1 sec
epoch 3, train loss 0.3464, train acc 0.874, test loss 0.3636, test acc 0.868, time 12.0 sec
epoch 4, train loss 0.3140, train acc 0.886, test loss 0.3495, test acc 0.873, time 13.0 sec
epoch 5, train loss 0.2902, train acc 0.895, test loss 0.3378, test acc 0.879, time 12.9 sec
epoch 6, train loss 0.2698, train acc 0.902, test loss 0.3256, test acc 0.878, time 12.0 sec
epoch 7, train loss 0.2527, train acc 0.908, test loss 0.3162, test acc 0.883, time 11.7 sec
epoch 8, train loss 0.2347, train acc 0.915, test loss 0.3143, test acc 0.887, time 11.6 sec
epoch 9, train loss 0.2214, train acc 0.920, test loss 0.3139, test acc 0.886, time 11.1 sec
epoch 10, train loss 0.2047, train acc 0.925, test loss 0.3063, test acc 0.891, time 11.1 sec
```